

Building Microservices with Structured Data Exchange

CIT 305 - IT31S1

Submitted by:

Ablola, Erika D.

Arbolente, Anne Patrice P.

Espino, Rose Carmen R.

Reyes, Janelle Renee I.

Submitted to:

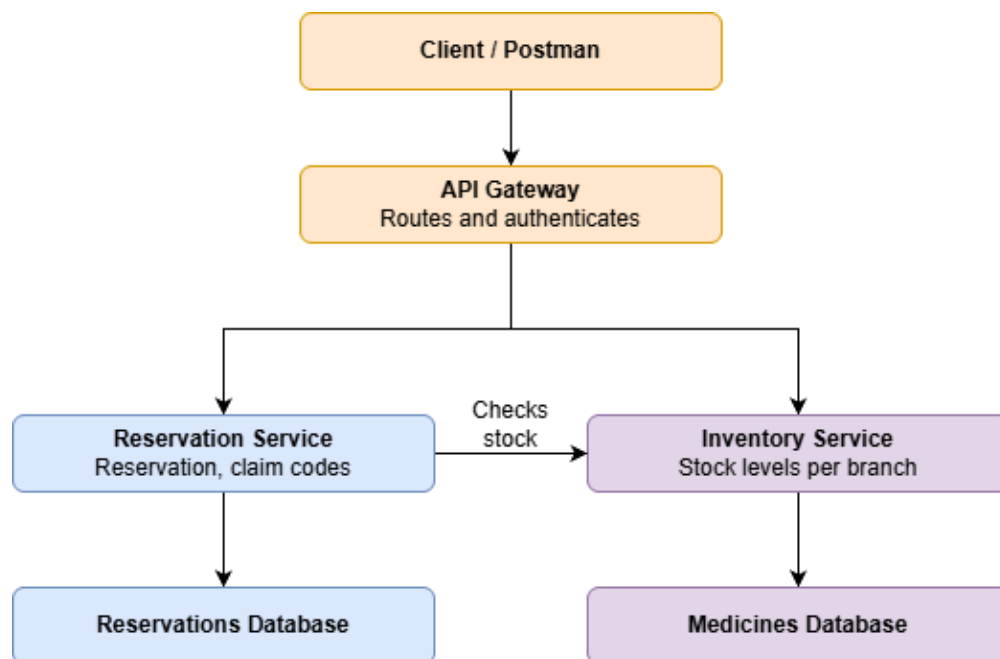
Ms. Justin Louise Neypes

September 8, 2026

I. Define Microservice Architecture

Item	Reservation Service (A)	Inventory Service (B)
Business Responsibility	Handles reservation creation, claim-code generation, and status or lifecycle tracking.	Tracks real-time medicine stock per branch and supplier info.
Main Resource / Data	Reservation includes is, medicine, quantity, branch, status, and claim code	Medicine or Inventory includes medicine id, name, stock quantity, and branch id.
Example Operations	Create reservation, check reservation status, update status (like approved, cancelled, or expired).	Get stock for a medicine, get all medicines for a branch, and update stock quantity.
Reason for Separation	Reservation logic changes independently including claim windows and status, and should not be blocked by inventory updates.	Inventory needs to be updated by many actors like staff, suppliers, and other reservations, keeping it separate avoids reservation logic owning stock data it does not control.

Diagram



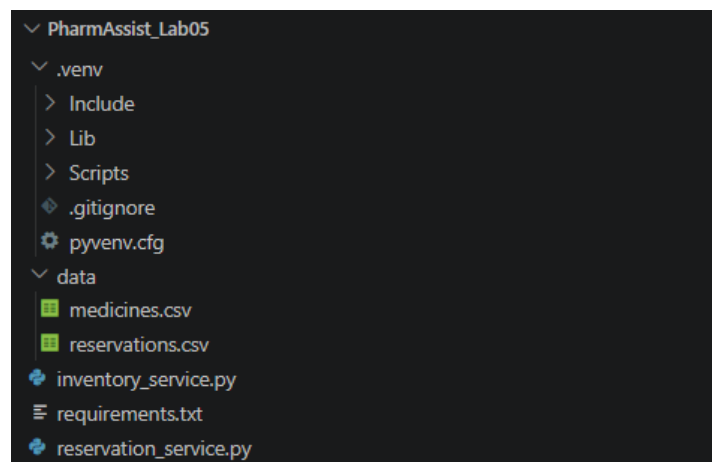
II. Design the REST API

SERVICE	METHOD	ENDPOINT	PURPOSE
Reservation Service	GET	/health	Checks whether reservation service is running
Reservation Service	GET	/reservations/{id}	Retrieves a specific reservation
Reservation Service	POST	/reservations	Creates a new reservation (checks stock via inventory service first)
Inventory Service	GET	/health	Checks whether inventory service is running
Inventory Service	GET	/medicines/{id}	Retrieves stock information for a specific medicine
Inventory Service	PUT	/medicines/{id}	Updates stock quantity

III. Implement the Microservices

For PharmAssist, the developers picked two independent running services: the Reservation Service and the Inventory Service.

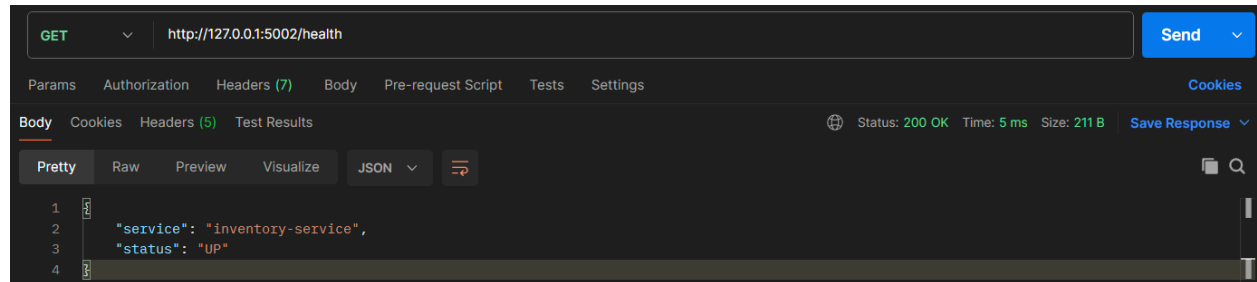
Service	Runs on Port	Main Function
Reservation Service	http://127.0.0.1:5001	Manages reservation records and processes medicine reservation requests
Inventory Service	http://127.0.0.1:5002	Manages medicine information, stocks, quantity, and stock status



Folder Structure in VS Code

```
(.venv) PS C:\Users\Raine\Documents\PharmAssist_Lab05> python .\reservation_service.py
* Serving Flask app 'reservation_service'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5001
Press CTRL+C to quit
```

Reservation Service running on port 5001

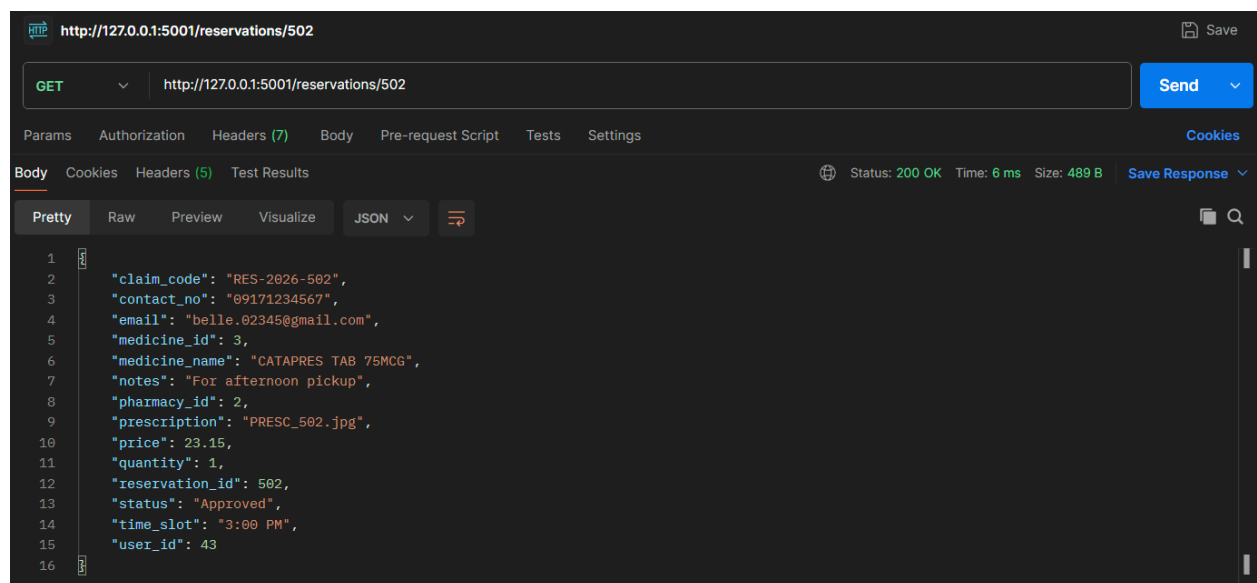


GET http://127.0.0.1:5002/health

Status: 200 OK Time: 5 ms Size: 211 B

Body: {"service": "inventory-service", "status": "UP"}

Reservation Service health check



GET http://127.0.0.1:5001/reservations/502

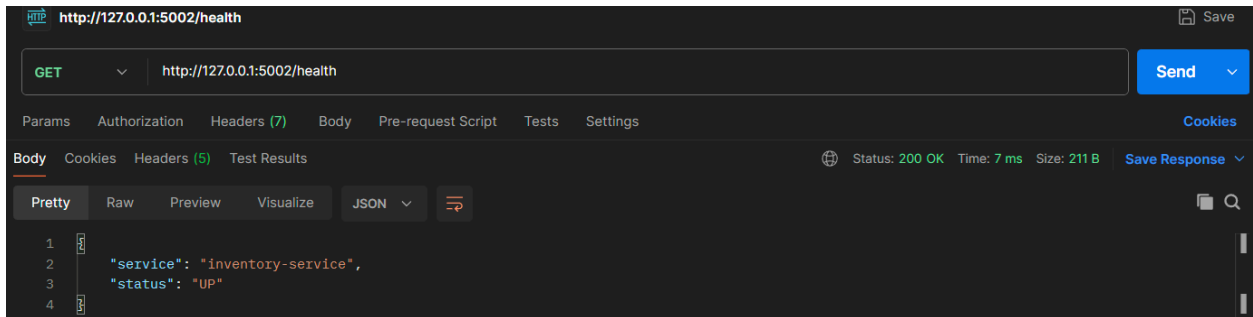
Status: 200 OK Time: 6 ms Size: 489 B

Body: {"claim_code": "RES-2026-502", "contact_no": "09171234567", "email": "belle.02345@gmail.com", "medicine_id": 3, "medicine_name": "CATAPRES TAB 75MCG", "notes": "For afternoon pickup", "pharmacy_id": 2, "prescription": "PRESC_502.jpg", "price": 23.15, "quantity": 1, "reservation_id": 502, "status": "Approved", "time_slot": "3:00 PM", "user_id": 43}

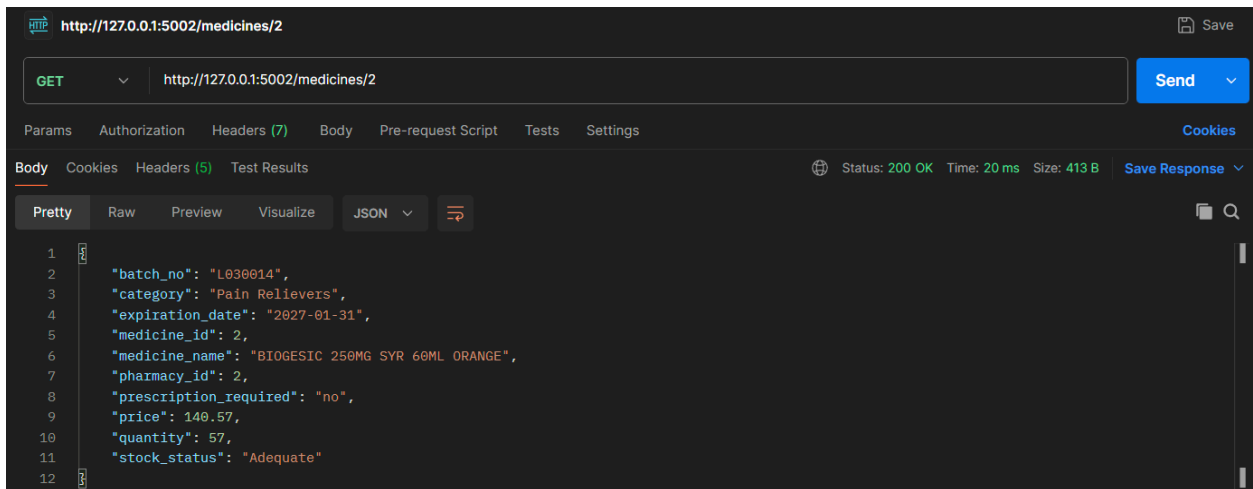
Reservation record retrieved from the Reservation Service

```
(.venv) PS C:\Users\Raine\Documents\PharmAssist_Lab05> python .\inventory_service.py
* Serving Flask app 'inventory_service'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5002
Press CTRL+C to quit
```

Inventory Service running on port 5002

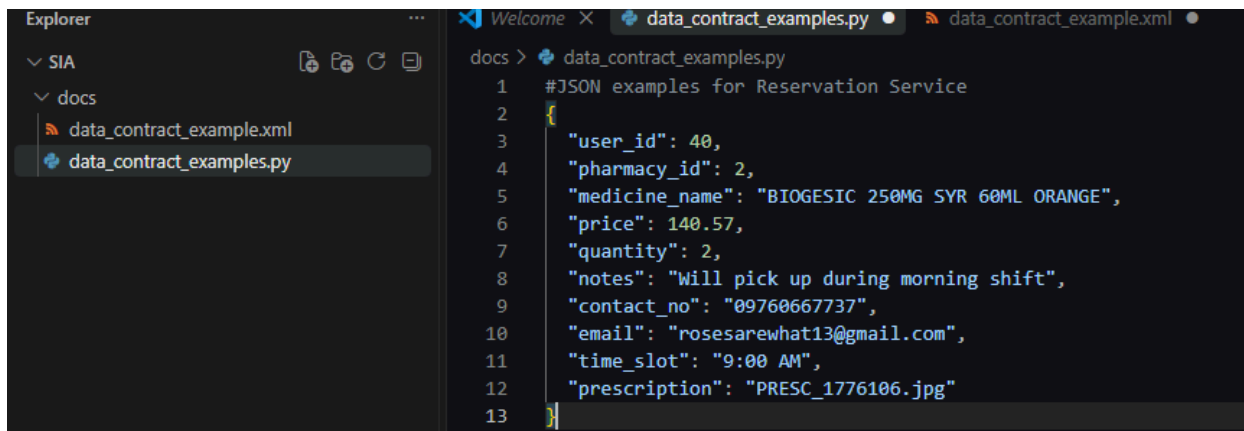


Inventory Service health check

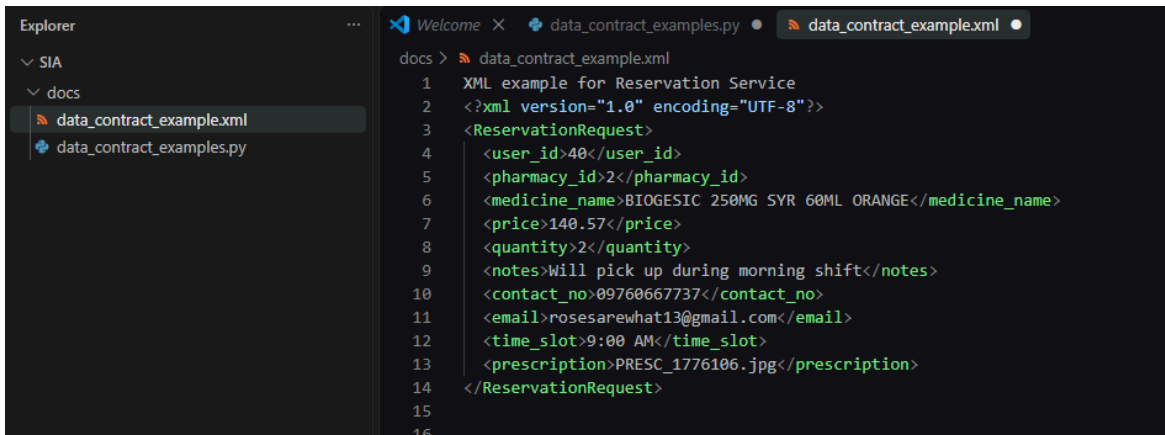


Inventory record retrieved from the Inventory Service

IV. Design the Data Contract

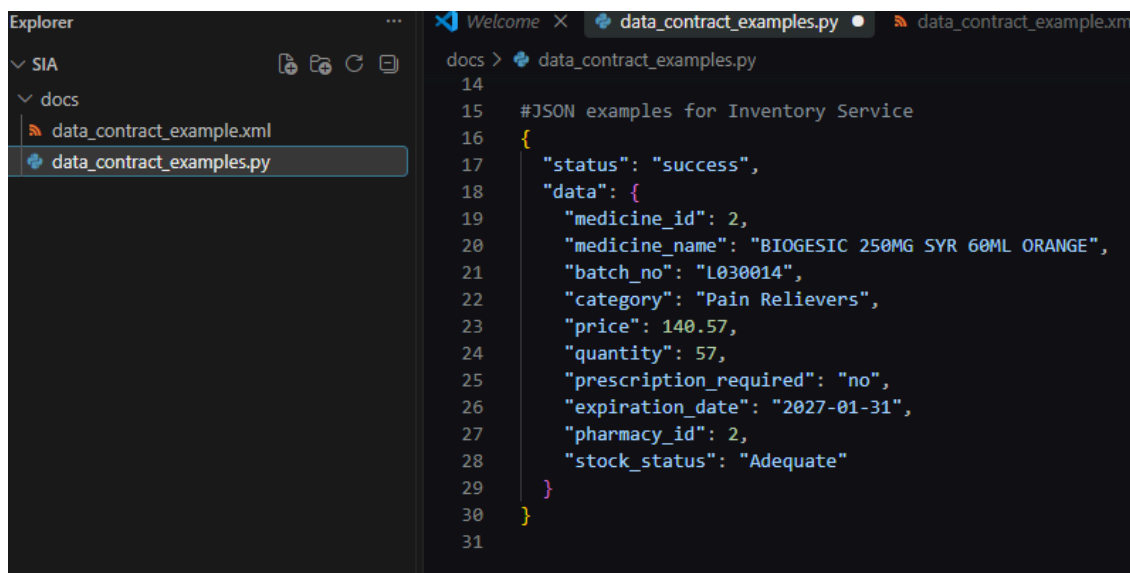


JSON example for reservation service



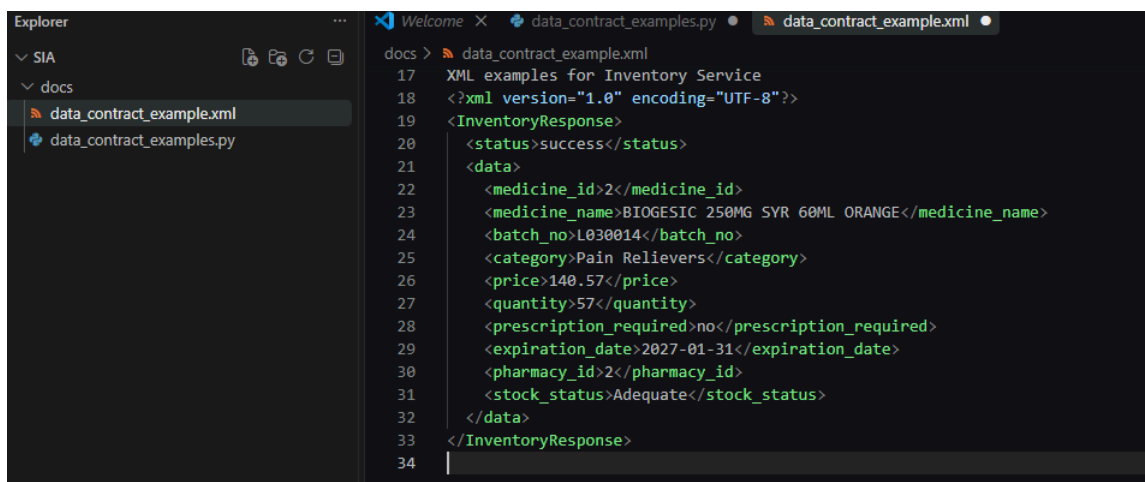
```
docs > data_contract_example.xml
1 XML example for Reservation Service
2 <?xml version="1.0" encoding="UTF-8"?>
3 <ReservationRequest>
4   <user_id>40</user_id>
5   <pharmacy_id>2</pharmacy_id>
6   <medicine_name>BIOGESIC 250MG SYR 60ML ORANGE</medicine_name>
7   <price>140.57</price>
8   <quantity>2</quantity>
9   <notes>Will pick up during morning shift</notes>
10  <contact_no>09760667737</contact_no>
11  <email>rosesarewhat13@gmail.com</email>
12  <time_slot>9:00 AM</time_slot>
13  <prescription>PRESC_1776106.jpg</prescription>
14 </ReservationRequest>
15
16
```

XML example for reservation service



```
docs > data_contract_examples.py
14
15 #JSON examples for Inventory Service
16 {
17   "status": "success",
18   "data": {
19     "medicine_id": 2,
20     "medicine_name": "BIOGESIC 250MG SYR 60ML ORANGE",
21     "batch_no": "L030014",
22     "category": "Pain Relievers",
23     "price": 140.57,
24     "quantity": 57,
25     "prescription_required": "no",
26     "expiration_date": "2027-01-31",
27     "pharmacy_id": 2,
28     "stock_status": "Adequate"
29   }
30 }
31
```

JSON example for inventory service



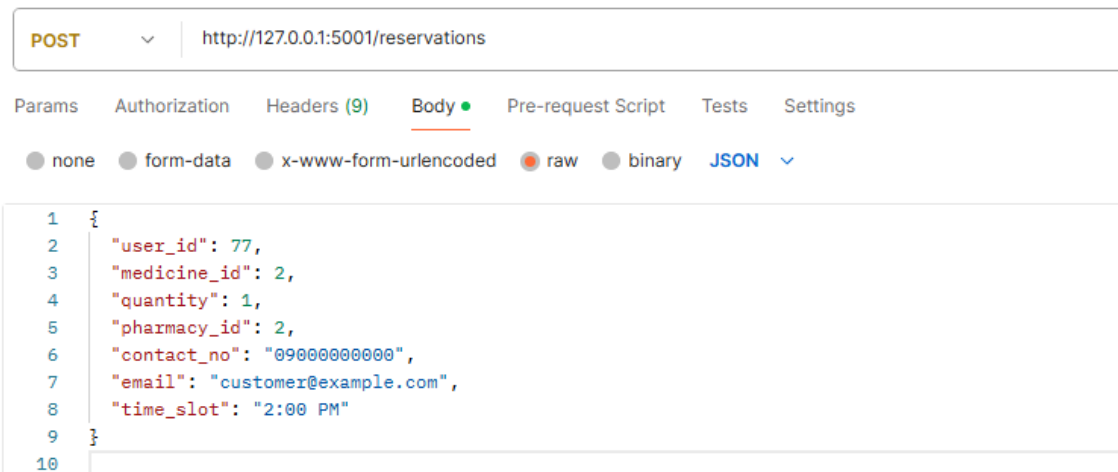
```
docs > data_contract_example.xml
17 XML examples for Inventory Service
18 <?xml version="1.0" encoding="UTF-8"?>
19 <InventoryResponse>
20   <status>success</status>
21   <data>
22     <medicine_id>2</medicine_id>
23     <medicine_name>BIOGESIC 250MG SYR 60ML ORANGE</medicine_name>
24     <batch_no>L030014</batch_no>
25     <category>Pain Relievers</category>
26     <price>140.57</price>
27     <quantity>57</quantity>
28     <prescription_required>no</prescription_required>
29     <expiration_date>2027-01-31</expiration_date>
30     <pharmacy_id>2</pharmacy_id>
31     <stock_status>Adequate</stock_status>
32   </data>
33 </InventoryResponse>
34
```

XML example for inventory service

V. Implement JSON Data Exchange

Endpoint: POST /reservations

Request: Customer submits a reservation as XML



Header: Content-Type: application/json

1. **JSON Parser** - This will verify the header and check if it's using application/json and then it also reads any incoming request from the body with request.get_json().

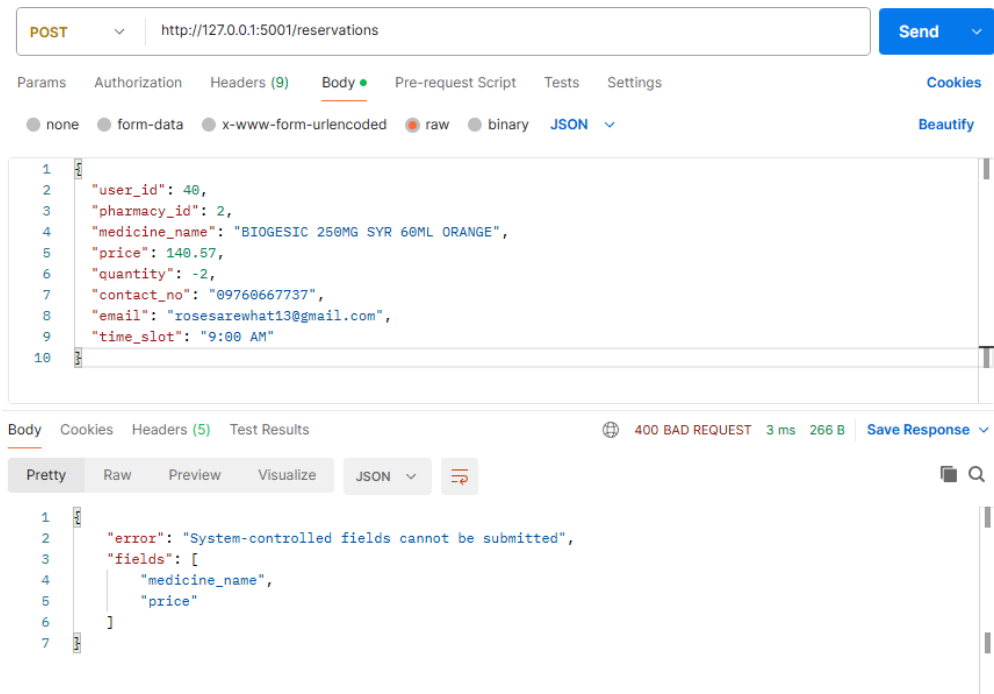
```
3 #JSON REQUEST: Accepts JSON requests for this endpoint
4 if not request.is_json:
5     return jsonify({
6         "error": "Unsupported Content-Type",
7         "message": "Use application/json"
8     }), 415
9
10 # DESERIALIZATION: Reads the JSON body sent by the customer
11 try:
12     data = request.get_json()
```

2. **Deserialization** - This code will get the reservation fields from the deserialized Python dictionary. It will then validate the required data types and any constraints before passing it to the business processing layer.

```
3 # DESERIALIZATION: Reads the JSON body sent by the customer
4 try:
5     data = request.get_json()
```

3. **Validation** - This is the code that will validate and ensure that the request body is really a JSON object.

```
#VALIDATION: Ensures the request body is a JSON object (dictionary)
if not isinstance(data, dict):
    return jsonify({
        "error": "Invalid request body"
    }), 400
```



The test case above will show that any input will be validated, and when it sees that the data input is wrong, the backend will intercept it and return an HTTP 400 Bad Request status along with the error message.

4. **Business Processing**

This service layer is the one that executes the business logic, verifying the stock available in the inventory service before creating a reservation record. It will compute the attributes, update the reservation data store, and then generate any appropriate statuses, such as an error response like 400 Bad Request.

5. Serialization

```
# SERIALIZATION: If the JSON structure is invalid, returns a 400 error
except BadRequest:
    return jsonify({
        "error": "Malformed JSON"
    }), 400

#VALIDATION: Ensures the request body is a JSON object (dictionary)
if not isinstance(data, dict):
    return jsonify({
        "error": "Invalid request body"
    }), 400
```

Body Cookies Headers (5) Test Results

Pretty

Raw

Preview

Visualize

JSON



```
1  {
2    "data": {
3      "claim_code": "RES-2026-505",
4      "contact_no": "09000000000",
5      "email": "customer@example.com",
6      "medicine_id": 2,
7      "medicine_name": "BIOGESIC 250MG SYR 60ML ORANGE",
8      "notes": "",
9      "pharmacy_id": 2,
10     "prescription": "",
11     "price": 140.57,
12     "quantity": 1,
13     "reservation_id": 505,
14     "status": "Pending",
15     "time_slot": "2:00 PM",
16     "user_id": 77
17   },
18   "message": "Reservation created successfully"
19 }
```

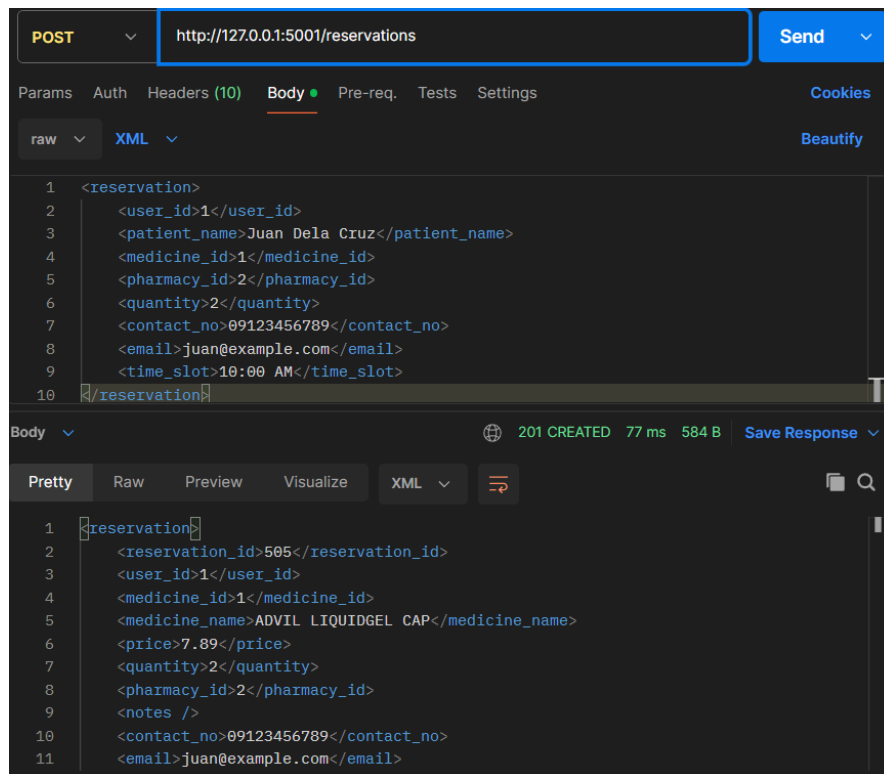
Header on response: Content-Type: application/json

The Postman execution above will show a successful POST request with a valid JSON body and header. It will return an HTTP 201 Created status with a serialized JSON response.

VI. Implement XML Data Exchange

Endpoint: POST /reservations

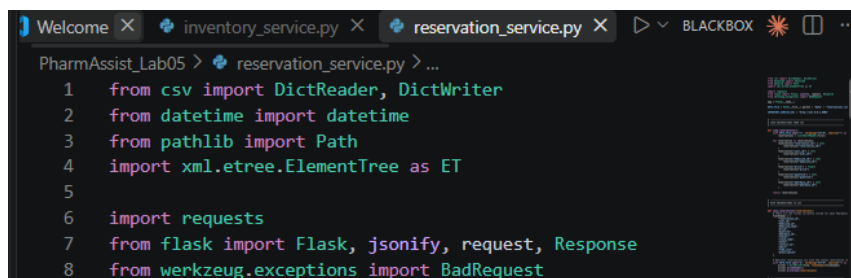
Request: Customer submits a reservation as XML



Header: Content-Type: application/xml

What the code needs to do:

1. **XML Parser** - read raw request body as XML. Since Python was chosen, `xml.etree.ElementTree` or the `xmltodict` library can be used.



```
# -----
# XML PARSING (Part 6)
# -----

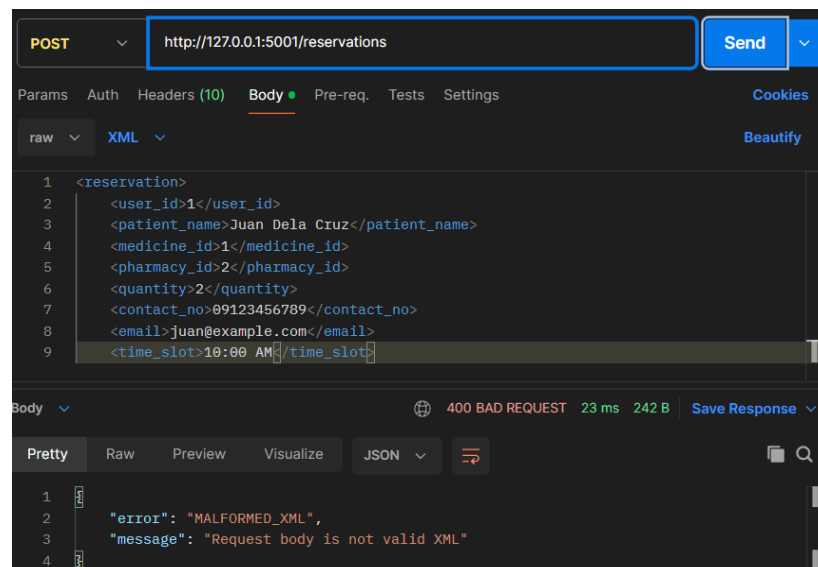
def xml_to_dict(xml_bytes):
    """Converts an incoming <reservation> XML body into a plain dict
    with the same shape as the JSON body PharmAssist already expects."""
    root = ET.fromstring(xml_bytes)
    return {child.tag: child.text for child in root}

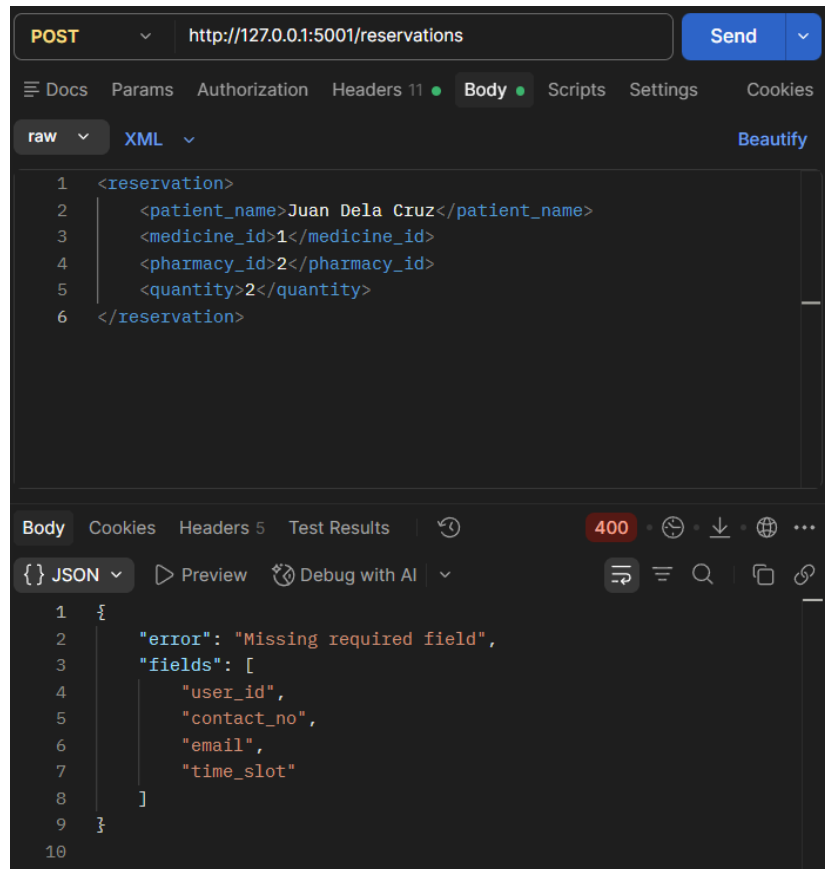
def reservation_to_xml(reservation):
    """Serializes a reservation dict back into XML for the response."""
    root = ET.Element("reservation")
    for key, value in reservation.items():
        element = ET.SubElement(root, key)
        element.text = str(value)
    return ET.tostring(root, encoding="unicode")
```

2. **Deserialization** - convert those XML elements into plain application data, a dict or object: {medicineId: 12, quantity: 2, branchId: 1}.

```
def xml_to_dict(xml_bytes):
    """Converts an incoming <reservation> XML body into a plain
    with the same shape as the JSON body PharmAssist already expects.
    root = ET.fromstring(xml_bytes)
    return {child.tag: child.text for child in root}
```

3. **Validation** - check medicineId, quantity, branchId are present and quantity > 0. Missing or Invalid is a 400 Bad Request.





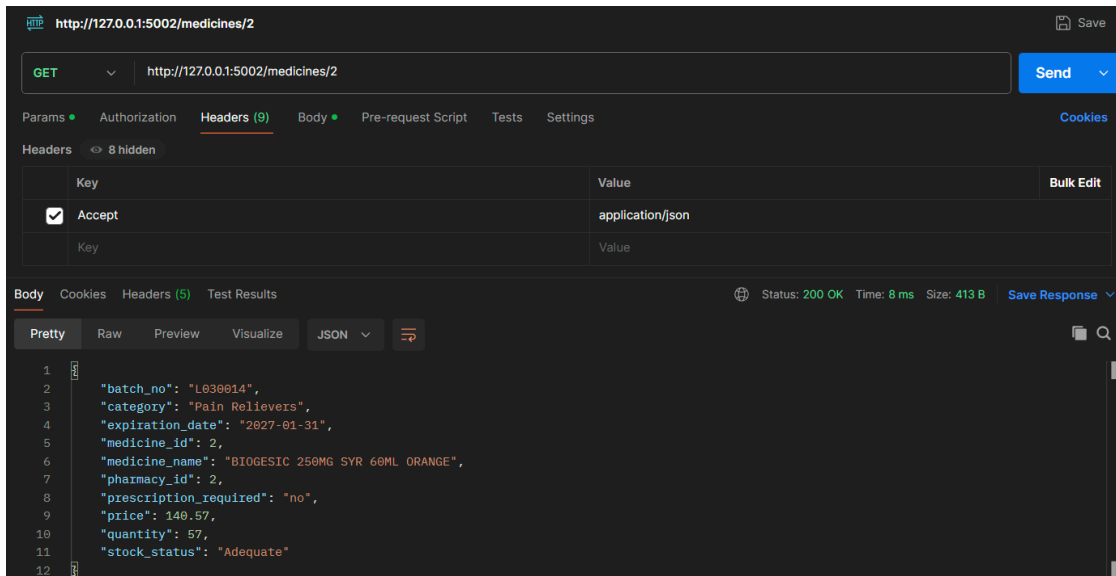
4. **Business Processing** - this is where the reservation actually gets created. Before saving it, the Reservation service checks medicine availability by calling the Inventory Service, only if enough stock exists does it proceed to generate reservationId, assign customer's request a status of "pending", and store the record.
5. **Serialize the result back to XML** - respond with the full reservation, which includes the fields the server generates:

```
def reservation_to_xml(reservation):
    """Serializes a reservation dict back into XML for the response."""
    root = ET.Element("reservation")
    for key, value in reservation.items():
        element = ET.SubElement(root, key)
        element.text = str(value)
    return ET.tostring(root, encoding="unicode")
```

Header on response: Content-Type: application/xml

VII. Implement Content Negotiation

For PharmAssist, the Inventory Service will use the `GET /medicines/{id}` endpoint to return the same medicine inventory record in either JSON or XML format. The response format will depend on the Accept header sent by the client. When the client sends `Accept: application/json`, the Inventory Service will return the medicine information in JSON format. On the other hand, when the client sends `Accept: application/xml`, the same medicine information will be returned but in XML format.

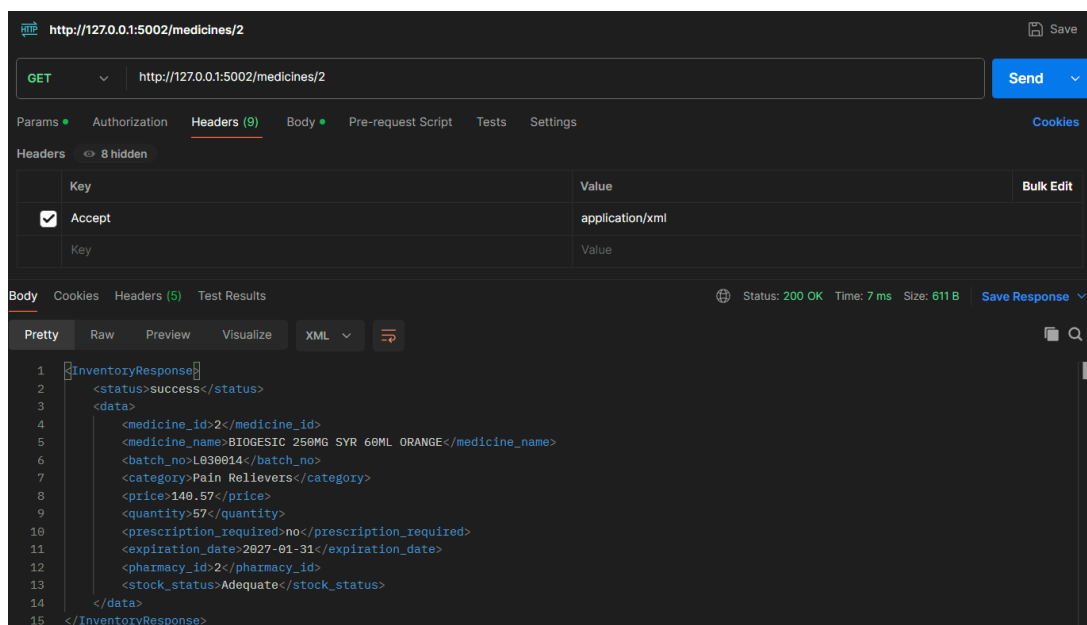


The screenshot shows a REST client interface with the following details:

- URL: `http://127.0.0.1:5002/medicines/2`
- Method: `GET`
- Headers (9):
 - `Accept: application/json`
- Body: A JSON object representing a medicine record.

```
1 {
2   "batch_no": "L838814",
3   "category": "Pain Relievers",
4   "expiration_date": "2027-01-31",
5   "medicine_id": 2,
6   "medicine_name": "BIOGESIC 250MG SYR 60ML ORANGE",
7   "pharmacy_id": 2,
8   "prescription_required": "no",
9   "price": 140.57,
10  "quantity": 57,
11  "stock_status": "Adequate"
12 }
```
- Status: `200 OK`, Time: `8 ms`, Size: `413 B`

Inventory Service response in JSON format



The screenshot shows a REST client interface with the following details:

- URL: `http://127.0.0.1:5002/medicines/2`
- Method: `GET`
- Headers (9):
 - `Accept: application/xml`
- Body: An XML document representing the same medicine record.

```
1 <InventoryResponse>
2   <status>success</status>
3   <data>
4     <medicine_id>2</medicine_id>
5     <medicine_name>BIOGESIC 250MG SYR 60ML ORANGE</medicine_name>
6     <batch_no>L838814</batch_no>
7     <category>Pain Relievers</category>
8     <price>140.57</price>
9     <quantity>57</quantity>
10    <prescription_required>no</prescription_required>
11    <expiration_date>2027-01-31</expiration_date>
12    <pharmacy_id>2</pharmacy_id>
13    <stock_status>Adequate</stock_status>
14  </data>
15 </InventoryResponse>
```
- Status: `200 OK`, Time: `7 ms`, Size: `611 B`

Inventory Service response in XML format

```

@app.get("/medicines/<int:medicine_id>")
def get_medicine(medicine_id):

    medicines = load_medicines()

    medicine = next(
        (
            medicine
            for medicine in medicines
            if medicine["medicine_id"]
               == medicine_id
        ),
        None
    )

    if medicine is None:
        return jsonify({
            "error": "Medicine not found"
        }), 404

    requested_type = (
        request.accept_mimetypes.best_match(
            [
                "application/json",
                "application/xml"
            ]
        )
    )

    if requested_type == "application/xml":

        xml_data = medicine_to_xml(
            medicine
        )

        return Response(
            xml_data,
            status=200,
            mimetype="application/xml"
        )

    return jsonify(medicine), 200

```

Inventory Service code for JSON and XML content negotiation

VIII. Implement Service-to-Service Integration

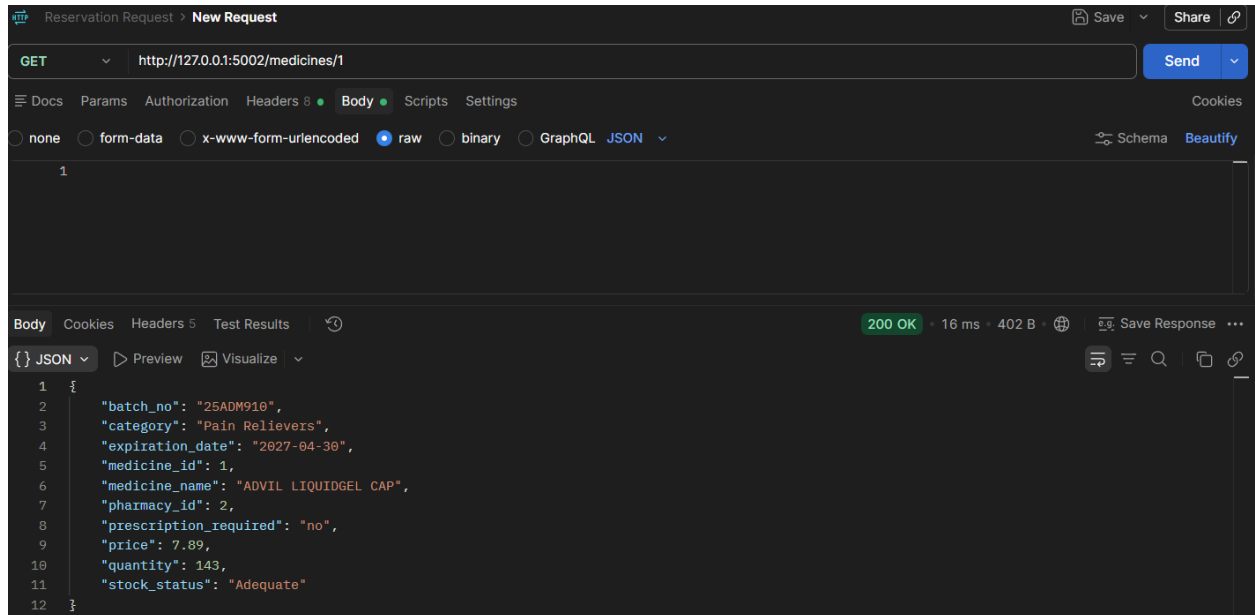
Business Transaction: Before the Reservation service can create a reservation, it must check with the Inventory service that enough stock exists.

Endpoint on Inventory Service (Service B) that Reservation service calls:

GET: /medicines/{medicineId}

Accept: application/json

Response:

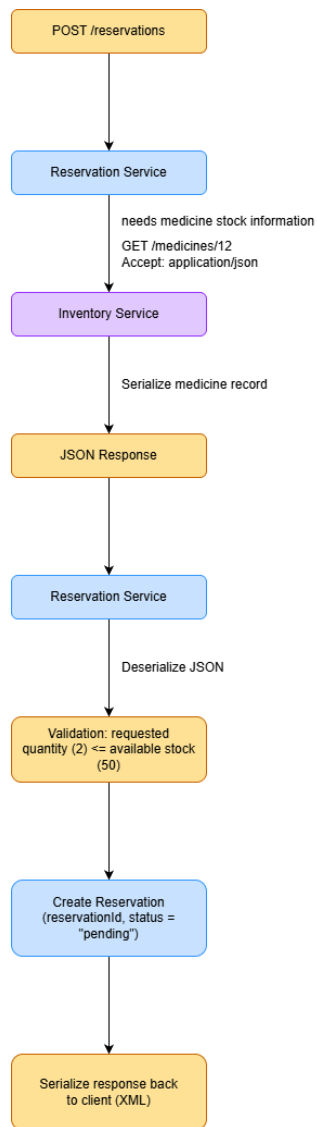


The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://127.0.0.1:5002/medicines/1
- Body:** raw
- Status:** 200 OK
- Time:** 16 ms
- Size:** 402 B
- Response Body (JSON):**

```
1 {
2   "batch_no": "25ADM910",
3   "category": "Pain Relievers",
4   "expiration_date": "2027-04-30",
5   "medicine_id": 1,
6   "medicine_name": "ADVIL LIQUIDGEL CAP",
7   "pharmacy_id": 2,
8   "prescription_required": "no",
9   "price": 7.89,
10  "quantity": 143,
11  "stock_status": "Adequate"
12 }
```

Full Flow



Two Failure Cases:

1. Not enough stock - 409 conflict

Request conflicts with current state, which is same idea as the “duplicate or conflicting operation”

POST http://127.0.0.1:5001/reservations

Body

```
1 <reservation>
2   <user_id>1</user_id>
3   <patient_name>Juan Dela Cruz</patient_name>
4   <medicine_id>1</medicine_id>
5   <pharmacy_id>2</pharmacy_id>
6   <quantity>99999</quantity>
7   <contact_no>09123456789</contact_no>
8   <email>juan@example.com</email>
9   <time_slot>10:00 AM</time_slot>
10 </reservation>
```

409

Body

```
1 {
2   "available_quantity": 143,
3   "error": "Insufficient stock",
4   "requested_quantity": 99999
5 }
```

2. Inventory service is down or unreachable - 503 Service Unavailable

POST http://127.0.0.1:5001/reservations

Body

```
1 <reservation>
2   <user_id>1</user_id>
3   <patient_name>Juan Dela Cruz</patient_name>
4   <medicine_id>1</medicine_id>
5   <pharmacy_id>2</pharmacy_id>
6   <quantity>2</quantity>
7   <contact_no>09123456789</contact_no>
8   <email>juan@example.com</email>
9   <time_slot>10:00 AM</time_slot>
10 </reservation>
```

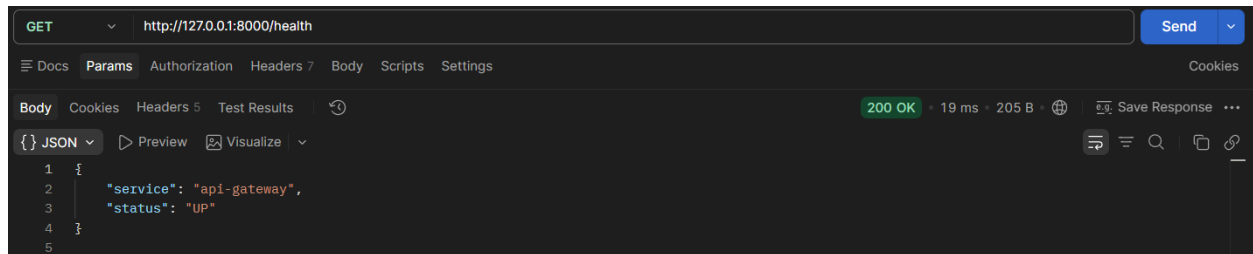
503

Body

```
1 {
2   "error": "Inventory Service unavailable"
3 }
```

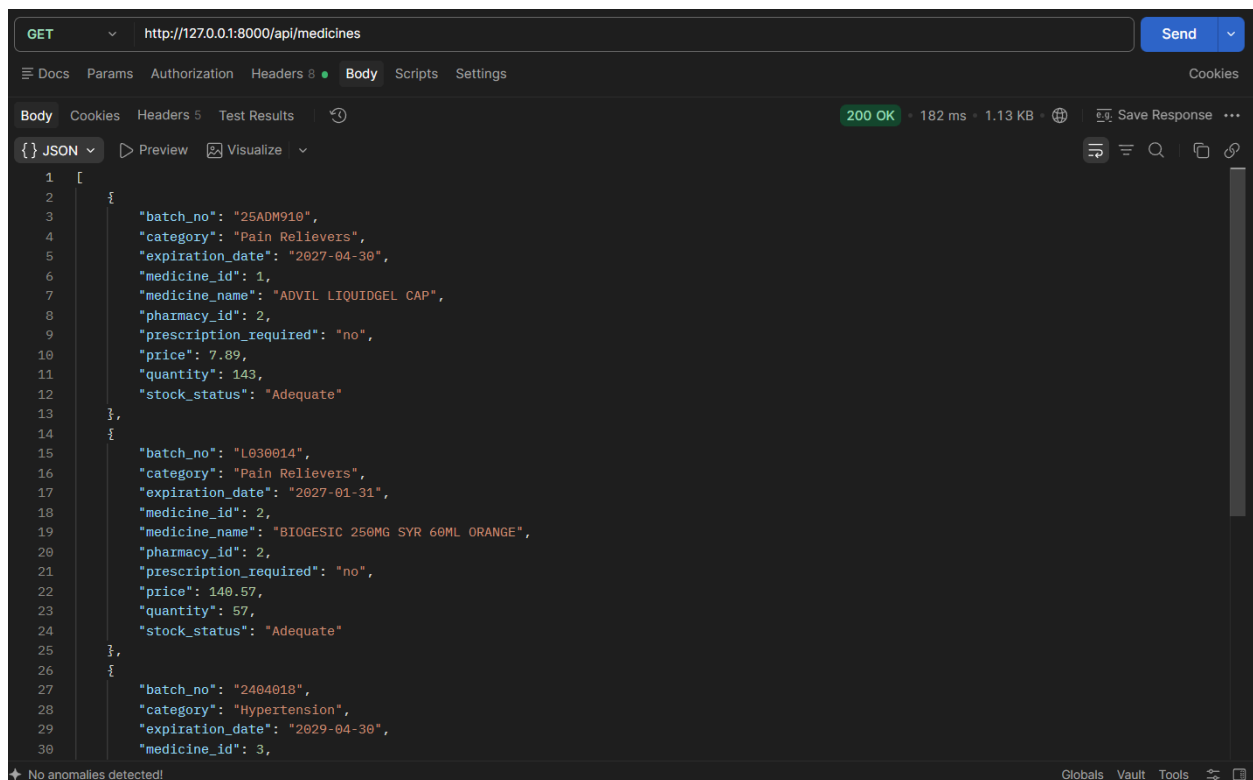
VIX. API Gateway/Centralized Routing

Incoming Path	Routed To	Service Base URL
/api/reservations/*	Reservation Service	http://localhost:5001
/api/medicines/*	Inventory Service	http://localhost:5002
/health	API Gateway	http://localhost:8000



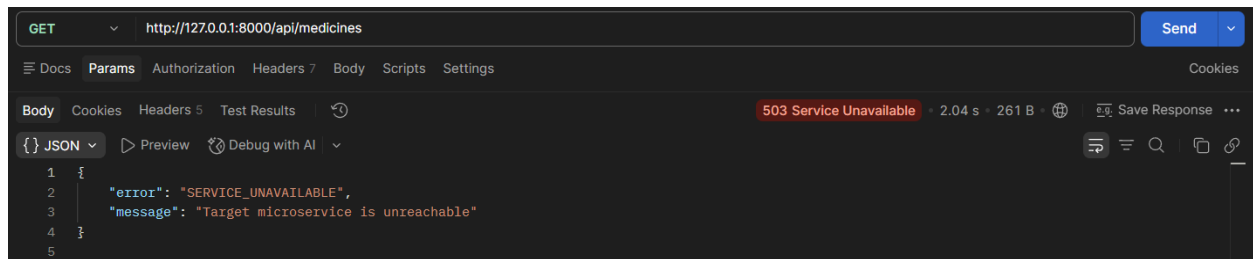
```
GET http://127.0.0.1:8000/health
200 OK - 19 ms - 205 B
{"service": "api-gateway", "status": "UP"}
```

Gateway Health Check



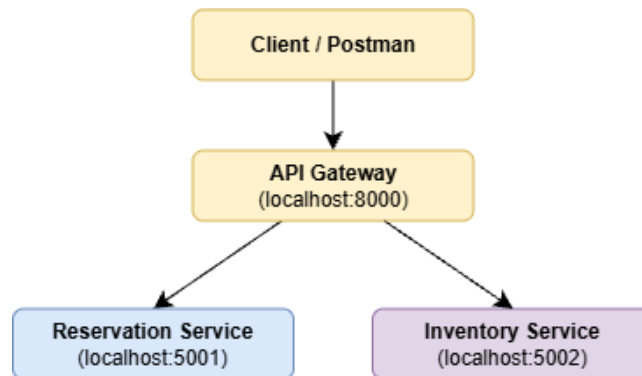
```
GET http://127.0.0.1:8000/api/medicines
200 OK - 182 ms - 1.13 KB
[{"batch_no": "25ADM910", "category": "Pain Relievers", "expiration_date": "2027-04-30", "medicine_id": 1, "medicine_name": "ADVIL LIQUIDGEL CAP", "pharmacy_id": 2, "prescription_required": "no", "price": 7.89, "quantity": 143, "stock_status": "Adequate"}, {"batch_no": "L030014", "category": "Pain Relievers", "expiration_date": "2027-01-31", "medicine_id": 2, "medicine_name": "BIOGESIC 250MG SYR 60ML ORANGE", "pharmacy_id": 2, "prescription_required": "no", "price": 140.57, "quantity": 57, "stock_status": "Adequate"}, {"batch_no": "2404010", "category": "Hypertension", "expiration_date": "2029-04-30", "medicine_id": 3, "medicine_name": "Lisinopril 10mg Tablets", "pharmacy_id": 1, "prescription_required": "yes", "price": 12.5, "quantity": 100, "stock_status": "Low"}]
```

Proxying or Routing Test



Fault Tolerance

Flow



Explanation:

- **Why the API Gateway exists** - Without the gateway, Postman would need to know which of two different ports handles which request. The gateway hides that: one address, one contract.
- **Which services it routes to** - Reservation service and Inventory service, matched purely by URL path prefix (/api/reservations/... vs /api/medicines/...).
- **How it simplifies client interaction** - the client only needs <http://localhost:8000> and never has to change its base URL even if you later move a service, add a third one, or deploy them to different hosts.

X. Validation and Error Handling

reservation_service.py		
Scenario	Code	Recommended HTTP Response
Malformed XML	<pre> if is_xml_request: try: data = xml_to_dict(request.data) except ET.ParseError: return jsonify({ "error": "MALFORMED_XML", "message": "Request body is not valid XML" }), 400 </pre>	400 Bad request
Malformed JSON	<pre> elif request.is_json: try: data = request.get_json() except BadRequest: return jsonify({ "error": "Malformed JSON" }), 400 </pre>	400 Bad request
Medicine not found	<pre> if inventory_response.status_code == 404: return jsonify({ "error": "Medicine not found" }), 404 </pre>	404 Not Found
Insufficient stock	<pre> # The reservation is not created if the requested quantity is > than the available stock if quantity > available_stock: return jsonify({ "error": "Insufficient stock", "message": "The requested quantity is greater than the available stock.", "requested_quantity": quantity, "available_quantity": available_stock }), 409 </pre>	409 Conflict
inventory_service.py		
Unsupported content type	<pre> if not request.is_json: return jsonify({ "error": "Unsupported Content-Type", "message": "Use application/json" }), 415 </pre>	415 Unsupported Media Type
Missing required field	<pre> if ("quantity" not in data or data["quantity"] in [None, ""]): return jsonify({ "error": "Missing required field", "field": "quantity" }), 400 </pre>	400 Bad Request

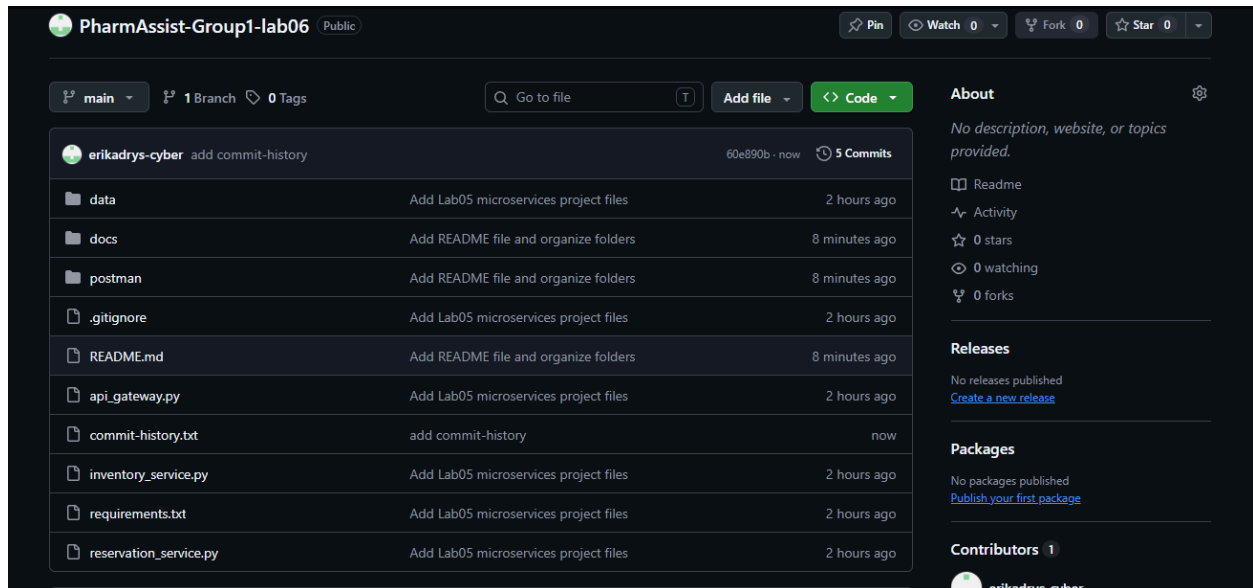
api_gateway.py		
Target microservice unreachable	<pre>except requests.exceptions.ConnectionError: return jsonify({"error": "SERVICE_UNAVAILABLE", "message": "Target microservice is unreachable"}), 503</pre>	503 Service Unavailable
Target microservice timeout	<pre>except requests.exceptions.Timeout: return jsonify({"error": "GATEWAY_TIMEOUT", "message": "Target microservice timed out"}), 504</pre>	504 Gateway Timeout

XI. Postman Integration Testing

For the Postman testing, the developers created a collection to organize and test the different API requests of the system. The collection contains the following test requests:

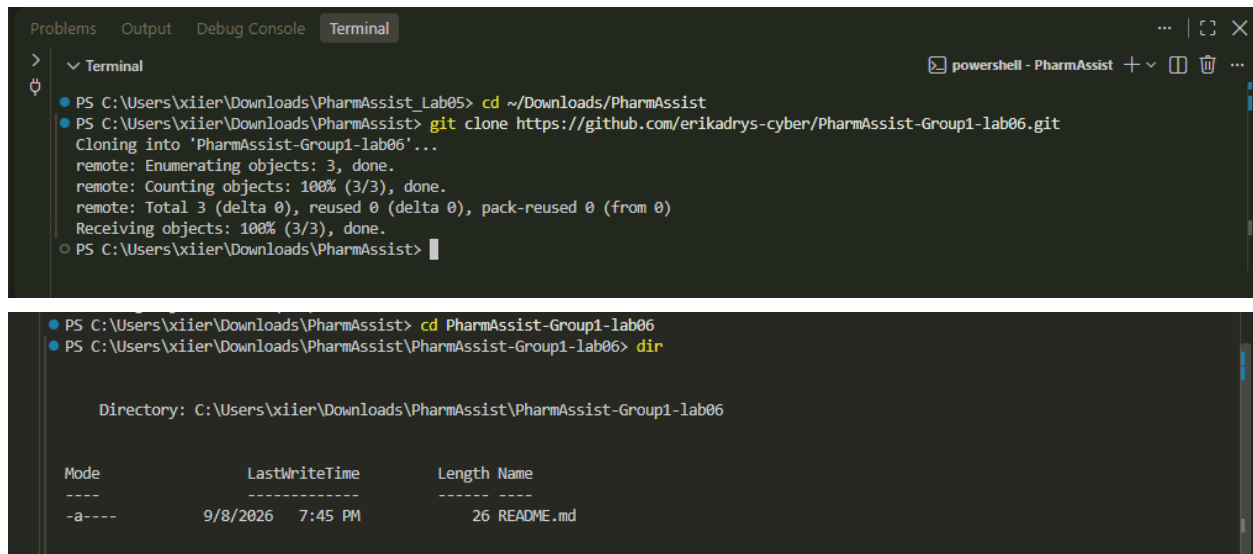
#	Postman Request	Test Result
01	Reservation Service Health	200 OK
02	Inventory Service Health	200 OK
03	Get Medicine - JSON	200 OK
04	Get Medicine - XML	200 OK
05	Create Reservation - JSON	201 Created
06	Create Reservation - XML	201 Created
07	Missing Required Field	400 Bad Request
08	Malformed JSON	400 Bad Request
09	Malformed XML	400 Bad Request
10	Reservation Not Found	400 Bad Request
11	Unsupported Media Type	415 Unsupported Media Type
12	Create Reservation - Integrated Transaction	201 Created
13	Create Reservation - Failed Integration	503 Service Unavailable

XII. GitHub Repository and Documentation



XIII. Git Commit History

Create repository on github, then connect it on the project files



Adding all the files on github

```
● PS C:\Users\xiier\Downloads\PharmAssist\PharmAssist-Group1-lab06> git add .
warning: in the working copy of 'PharmAssist-Lab-06-Integration-Testing.postman_collection.json', LF will be replaced by CRLF the next time Git touches it
● PS C:\Users\xiier\Downloads\PharmAssist\PharmAssist-Group1-lab06> git commit -m "Add Lab05 microservices project files"
[main aeb5735] Add Lab05 microservices project files
 8 files changed, 1995 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 PharmAssist-Lab-06-Integration-Testing.postman_collection.json
 create mode 100644 api_gateway.py
 create mode 100644 data/medicines.csv
 create mode 100644 data/reservations.csv
 create mode 100644 inventory_service.py
 create mode 100644 requirements.txt
 create mode 100644 reservation_service.py
```

```
● PS C:\Users\xiier\Downloads\PharmAssist\PharmAssist-Group1-lab06> git push
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 8 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (11/11), 11.09 KiB | 757.00 KiB/s, done.
Total 11 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/erikadrys-cyber/PharmAssist-Group1-lab06.git
 01bedd0..aeb5735  main -> main
```

Adding the document examples

```
nothing added to commit but untracked files present (use "git add" to track)
● PS C:\Users\xiier\Downloads\PharmAssist\PharmAssist-Group1-lab06> git add .
● PS C:\Users\xiier\Downloads\PharmAssist\PharmAssist-Group1-lab06> git commit -m "Add docs/ files"
[main 24f5b55] Add docs/ files
 2 files changed, 63 insertions(+)
 create mode 100644 docs/examples/data_contract_example.xml
 create mode 100644 docs/examples/data_contract_examples.py
● PS C:\Users\xiier\Downloads\PharmAssist\PharmAssist-Group1-lab06> git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 1.27 KiB | 216.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/erikadrys-cyber/PharmAssist-Group1-lab06.git
 aeb5735..24f5b55  main -> main
```

Adding [README.md](#) file and microservices architecture

```
● PS C:\Users\xiier\Downloads\PharmAssist\PharmAssist-Group1-lab06> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    deleted:   PharmAssist-Lab-06-Integration-Testing.postman_collection.json
    modified:   README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    docs/architecture/
    postman/
```

```
● PS C:\Users\xiier\Downloads\PharmAssist\PharmAssist-Group1-lab06> git add .
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'postman/PharmAssist-Lab-06-Integration-Testing.postman_collection.json', LF will be replaced by CRLF the next time Git touches it
● PS C:\Users\xiier\Downloads\PharmAssist\PharmAssist-Group1-lab06> git commit -m "Add README file and organize folders"
[main c9ac10c] Add README file and organize folders
3 files changed, 364 insertions(+), 1 deletion(-)
create mode 100644 docs/architecture/microservice_architecture.png
rename PharmAssist-Lab-06-Integration-Testing.postman_collection.json => postman/PharmAssist-Lab-06-Integration-Testing.postman_collection.json (100%)
● PS C:\Users\xiier\Downloads\PharmAssist\PharmAssist-Group1-lab06> git push
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 29.09 KiB | 4.85 MiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
```